Desk Number _____

Student Number _____

Student Name _____

School of Data Science

FINAL EXAMINATION

Term 2 2025-2026

CSC3150 Operating System

Examination Duration: 120 minutes

This examination has 8 questions.

The score distribution of the 8 question is Question 1 (10%), Question 2 (8%), Question 3 (16%), Question 4 (16%), Question 5 (12%), Question 6 (22%), and Question 7 (10%) and Question 8 (6%).

Exam Conditions:

This is a FORMAL Examination

This is a RESTRICTED OPEN BOOK Exam. Maximum of one (1) sheet of handwritten notes double-sided are permitted

Materials Permitted:

Maximum of one (1) sheet of A4-sized double-sided notes are permitted. **NO OTHER MATERIALS PERMITTED**

Calculators without the functionalities of programming and file storage are permitted.

Materials Supplied:

1 × 6 Page Answer Booklet

You must write all your answers in your answer book.

Question 1. Concepts and definitions. (10 points)

1.1 (2 points) Explain the difference between a process and a thread. Give one advantage of threads over processes and one disadvantage.

Thread is the single unique execution context in CPU, while Process often contains thread(s) and has distinct address space between each other.

Thread has cheaper communication overhead but is prone to Race Conditions.

1.2 (2 points) What is dual mode in an operating system? What is its purpose?

Dual mode means kernel mode and user mode. To isolate faults and protect the OS.

1.3 (2 points) What is a race condition? Give a simple example code.

global int c = 0 Thread A: c++ Thread B: c++

Running the code gives c = 1 or 2 due to interleaving / missed update.

1.4 (4 points) Consider the following Pthreads interface:

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void*), void *arg);
void pthread_exit(void *value_ptr);
int pthread_join(pthread_t thread, void**value_ptr);
```

Ignoring attr and start_routine, explain the pointer design of the following parameters:

- why thread in pthread_create is a pointer
- why arg in pthread_create is a pointer
- why value_ptr in pthread_exit is a void *
- why value_ptr in pthread_join is a void **

Because pthread_create writes the newly created thread ID into the caller's variable;

arg is a generic pointer used to pass user-defined data to the thread's start routine;

void * allows a thread to return a generic pointer value that can be retrieved by another thread through pthread_join;

void ** allows pthread_join to write the exiting thread's void * return value into the caller's pointer variable.

Question 2. System programming and OS foundations. (8 points)

A program creates a child process using fork(). Immediately after the fork(), both parent and child execute the same next line of code.

2.1 (3 points) Answer the following questions.

- What values are returned by fork() in the parent and child?

Parent receives child's PID which > 0; Child returns 0.

- What memory is logically duplicated?

Virtual address space, e.g. data, stack, heap, code etc.

- Name one thing that is shared and one thing that is not shared.

Their fd tables refer to the same OFDs. Memory space is separated.

2.2 (3 points) A developer wants two unrelated processes on the same machine to communicate. For each of the following IPC mechanisms, say whether it is appropriate; if not, explain why:

- Pipe: No, since pipe usually works for related processes through inherited fd;
- Socket: Yes, doesn't need to be related and works on local/network processes;
- Shared memory: Yes, but requires explicit synchronization.

2.3 (2 points) Why does the OS expose files as an abstraction? Name two benefits of the file abstraction from the perspective of application programmers and OS designers.

For OS designers, its more portable, i.e. no need to redesign OS interface when changing Hardware.

For programmers, e.g. we can have sorted and easy-to-use directory/folder abstraction to store data without needing to manage raw disk blocks.

Question 3. Synchronization. (16 points)

A bounded buffer has size N. Multiple producers call put(Item), and multiple consumers call get(). Assume the following semaphore and lock primitives are available:

- wait(s) and signal(s) for semaphores
- lock(m) and unlock(m) for mutexes

You may assume:

- semaphore empty counts the number of empty slots
- semaphore full counts the number of full slots

- mutex m protects the shared buffer state

The shared data structure is:

```
Item buf[N];
int head = 0, tail = 0;
semaphore empty = N;
semaphore full = 0;
mutex m;
```

3.1 (6 points) Fill in the code for put(Item) and Item get() using semaphores and a mutex.

Your solution must:

- correctly block producers when the buffer is full
- correctly block consumers when the buffer is empty
- correctly update head and tail
- use the synchronization primitives in the correct order

```
void put(Item x) {
    // fill in Producer
    wait(empty);
    lock(m);
    buf[tail] = x;
    tail = (tail + 1) % N;
    unlock(m);
    signal(full)
}

Item get() {
    Item x;
    // fill in Consumer
    wait(full);
    lock(m);
    x = buf[head];
    head = (head + 1) % N;
    unlock(m);
    signal(empty);
    return x;
}
```

3.2 (4 points) Suppose a programmer removes the mutex and keeps only empty and full. Is this correct? Why? if not correct, give a concrete example of what can go wrong.

No. E.g. Consider when tail is 0, so empty slots are enough, and 2 producers gets into the original critical section due to switching, they read the same tail index and write to the same slot, causing lost update/missed update.

3.3 (3 points) Suppose a programmer swaps the order of wait(empty) and lock(m) inside put(). Will this lead to a problem, and why?

Yes. May cause circular wait, i.e. a deadlock: If empty is 0, and put() acquires lock, then get() cannot operate on buffer, so empty cannot be increased to positive value, thereby causing put() to wait indefinitely.

3.4 (3 points) Suppose a programmer writes the following incorrect producer code:

```
void put(Item x) {
    wait(empty);
    buf[tail] = x;
    tail = (tail + 1) % N;
    signal(full);
}
```

Assume there are multiple producers and consumers. Explain what synchronization is missing and give one concrete problem that may occur.

The code is missing MUTEX. Although empty and full control the number of available slots, they do not protect shared buffer state. Two producers may simultaneously read the same tail value, overwrite each other's items, and incorrectly update tail.

Question 4. CPU scheduling with multi-level feedback queue. (16 points)

Consider a multilevel feedback queue scheduler with the following rules:

- There are 3 queues: Q0, Q1, Q2
- New jobs enter Q0
- Q0 has time quantum = 2
- Q1 has time quantum = 3
- Q2 runs FCFS
- If a job uses up its entire quantum in Q0 or Q1, it is demoted one level
- If a job yields the CPU before its quantum expires, it stays at the same level
- Higher-priority queues always run before lower-priority queues (with pre-emption)
- There is no priority boost for the sake of simplicity

Now, three jobs arrive at time 0, 1, 2, in the order A, B, C, respectively:

Job	CPU behavior
A	Needs 6 units of CPU, never blocks
B	Needs 3 units of CPU, never blocks
C	Executes 1 unit of CPU, then blocks for I/O for 2 unit of time; then executes 1 unit of CPU, blocks again for 2 unit of time; then executes 1 final unit of CPU and terminates

Assume that when C finishes I/O, it returns to the same queue it was in when it blocked.

4.1 (9 points) Simulate the execution until all jobs complete. Fill in the following trace.

Time interval	Which job is running	At which queue
0-1	A	Q0
1-2	A	Q0
2-3	B	Q0
3-4	B	Q0
4-5	C	Q0
5-6	A	Q1
6-7	A	Q1
7-8	C	Q0
8-9	A	Q1
9-10	B	Q1
10-11	C	Q0
11-12	A	Q2

Then answer:

- At what times is a job demoted?

At time 1-2, A is demoted; then B demoted at 3-4; and A demoted again from Q1 at time 8-9.

- At what time does C return from I/O?

7-8 and 10-11.

- At what time does each job finish?

A: 11-12, B: 9-10, C: 10-11

4.2 (3 points) What is the waiting time for each job?
A: 12-6= 6, B: 10-3-1= 6, C: 11-3-4-2=2

4.3 (2 points) What feature of MLFQ is intended to favor interactive jobs?
The tendency of demoting CPU-bound jobs and keeping I/O-bound (interactive) jobs at high priority queues.

4.4 (2 points) Give one workload or scenario in which MLFQ behaves poorly or unfairly.
MLFQ can be misled, e.g. if CPU-bound jobs launch meaningless I/O such as printf() before the end of its time quantum will make MLFQ's rule inefficient thus works poorly.

Question 5. Deadlock. (12 points)

Part A: Single-instance resources
A system has:

- processes: P1, P2, P3
- resource types: R1, R2, R3
- one instance of each resource

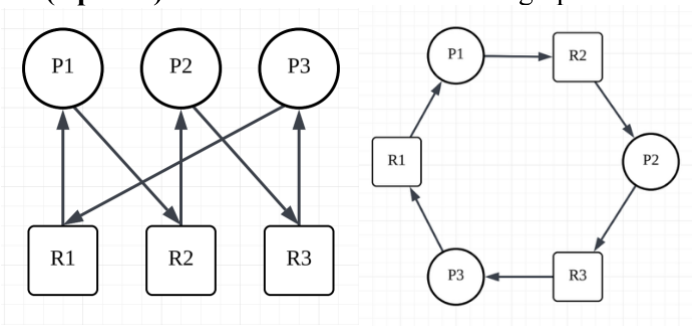
Current allocation:

- R1 is allocated to P1
- R2 is allocated to P2
- R3 is allocated to P3

Current outstanding requests:

- P1 requests R2
- P2 requests R3
- P3 requests R1

5.1 (4 points) Draw the resource-allocation graph.



Both of above are acceptable.

5.2 (2 points) Is the system deadlocked? Justify your answer using the graph.

Yes, as a single-instance RAG, observing a cycle means the system is definitely Deadlocked.

5.3 (2 points) If the system is deadlocked, describe one concrete way to resolve it.

Break the "No Pre-emption" condition of the 4 Coffman Conditions.

E.g.1 Allowing OS to pre-empt or take back some resources by force, such as paging out to disc for certain Memory resources.

E.g.2 The OS can forcibly pre-empt R1 from P1 and give it to P3, allowing P3 to complete and release R3. Then P2 and P1 can finish.

Part B: Multiple-instance resources

A second system has:

- resource type A with 2 instances
- resource type B with 1 instance

Its current state is:

Process	Allocation	Request
P1	A=1, B=0	A=0, B=1
P2	A=1, B=0	A=0, B=1
P3	A=0, B=1	A=1, B=0

5.4 (2 points) Run the deadlock detection algorithm manually and state clearly whether the system is deadlocked.

Deadlocked because all Processes require extra resources to finish while FreeResources = [0, 0].

5.5 (2 points) If the system is deadlocked, give one way to resolve it. If the system is not deadlocked, describe one additional request or change of state that would put it into deadlock.

OS mandates P3 to give up hold on B (if B is not mutex lock!), then P1 or P2 can finish, breaking the deadlock.

(The rest are out of range for midterm.)

Question 6. Memory Management (22 points) Out of Syl.

Assume a demand-paging system with 3 physical frames and the page reference string:
A, B, C, A, D, B, A, E, B, A, C

6.1 (11 points) For the reference string above, compute the number of page faults under FIFO and LRU, and MIN respectively. Show the frame contents after each reference and answer the number of page faults.

FIFO:

Page\Ref	A	B	C	A	D	B	A	E	B	A	C
1	A										
2		B									
3			C								

Total number of page fault:

LRU:

Page\Ref	A	B	C	A	D	B	A	E	B	A	C
1	A										
2		B									
3			C								

Total number of page fault:

6.2 (7 points) Apply the Clock replacement algorithm to the same reference string. Assume the following Clock-algorithm conventions:

- There are 3 physical frames, arranged in a circle.
- Each frame has a reference bit (R bit).
- When a page is referenced and is already in memory, its R bit is set to 1.
- On a page fault, the clock hand examines frames starting from its current position:
  - if the current frame has R = 1, set it to 0 and advance the hand to the next frame;
  - if the current frame has R = 0, replace that page with the new page.
- After a new page is loaded, its R bit is set to 1, and the clock hand advances to the next frame.
- Initially, all frames are empty, and the clock hand starts at frame 0.
- Count one hand movement each time the clock hand advances by one frame.
- A full cycle means the clock hand makes one complete revolution over all 3 frames.

Please fill in the table below and answer questions afterwards:

Ref	Page fault?	Frames after step	R bits after step	Hand after step
A	Yes	[A, -, -]	[1, -, -]	1
B	Yes	[A, B, -]	[1, 1, -]	2

C	Yes	[A, B, C]	[1, 1, 1]	0
A				
D				
B				
A				
E				
B				
A				
C				

- The total page fault number is:
- The number of total full cycles that the clock hand has traversed:

6.3 (2 points) Explain the idea of multi-level paging. Why is it preferable to storing one large flat page table for a large sparse virtual address space?

6.4 (2 points) Suppose the system has the following timing parameters:

- TLB lookup time = 10 ns
- Cache access time = 5 ns
- Main memory access time = 100 ns
- TLB hit rate = 90%
- Cache hit rate = 80%

Assume:

- on a TLB hit, address translation costs one TLB lookup, then the data is accessed through the cache
- on a TLB miss, the system must first access memory to obtain the page-table entry, then access the data through the cache
- if the data is found in cache, no main-memory access is needed for the data ignore page faults

Please compute the effective access time for (a) TLB hit + cache hit, (b) TLB hit + cache miss, (c) TLB miss + cache hit, (d) TLB miss + cache miss.

Question 7. Files and file systems (10 points) Out of Syl. Except for 7.1

7.1 (4 points) Explain the difference between high-level file I/O and low-level file I/O. Give one advantage of each.

High-level file I/O uses library functions (e.g., fopen, fprintf) with buffering and formatted I/O, while low-level file I/O uses system calls (e.g., open, read, write) operating on file descriptors. High-level I/O is easier to use for programmers and reduces system call overhead through buffering. Low-level I/O provides more direct control over file descriptors and I/O behavior.

7.2 (4 points) Compare Unix inode-based indexed allocation and FAT-style linked allocation. For each file system, explain (a) how file blocks are located, (b) one advantage, and (c) one disadvantage.

7.3 (2 points) A journaling file system writes metadata updates to a journal before applying them to the main file system. Answer the following:

- What problem is journaling trying to solve?

- After a crash, what does recovery do with the journal?

Question 8. Distributed Systems (6 points) **Out of Syl.**

8.1 (3 points) The lecture states that NFS uses weak consistency. Explain what this means. In your answer:

- describe how clients learn that a file has changed
- explain why a client may temporarily read stale data
- explain what may happen if multiple clients write the same file concurrently

8.2 (3 points) Consider two-phase commit (2PC) with one coordinator and several participants. For each of the following failure scenarios, state what happens and whether any process may become blocked.

(a) The coordinator crashes before sending VOTE-REQ to all participants.
(b) All participants vote YES, but the coordinator crashes before sending the final decision.
(c) A participant has already received GLOBAL-COMMIT, while another participant is still waiting when the coordinator crashes.

END OF EXAMINATION